Date: 29-12-2025 à Day 2 – Querying & Modifying Data

By Shaaz

1. Create Database command.

```
CREATE DATABASE InsuranceDB
```

2. Create table commands for all the tables with constraints, relationships etc.

```
CREATE TABLE Customers (
   CustomerID INT IDENTITY PRIMARY KEY,
   FirstName VARCHAR(20) NOT NULL,
   LastName VARCHAR(20) ,
   DateOfBirth DATE NOT NULL,
   Phone VARCHAR(15) NOT NULL,
   Email VARCHAR(50) NOT NULL UNIQUE
);


CREATE TABLE Agents (
   AgentId INT IDENTITY PRIMARY KEY,
   AgentName VARCHAR(20) NOT NULL,
   Phone VARCHAR(15) NOT NULL,
   City VARCHAR(20) NOT NULL
);


CREATE TABLE Policies (
   PolicyId INT IDENTITY PRIMARY KEY,
   PolicyName VARCHAR(20) NOT NULL,
   PolicyType VARCHAR(20) NOT NULL,
   PremiumAmount MONEY NOT NULL,
   DurationYears INT NOT NULL
);


CREATE TABLE PolicyAssignments (
   AssignmentID INT IDENTITY PRIMARY KEY,
   CustomerID INT NOT NULL,
   PolicyId INT NOT NULL,
```

```
    AgentId INT NOT NULL,
    StartDate DATE NOT NULL,
    EndDate DATE NOT NULL,

    CONSTRAINT FK_PA_Customers
        FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),

    CONSTRAINT FK_PA_Policies
        FOREIGN KEY (PolicyId) REFERENCES Policies(PolicyId),

    CONSTRAINT FK_PA_Agents
        FOREIGN KEY (AgentId) REFERENCES Agents(AgentId)
);


CREATE TABLE Claims (
    ClaimsID INT PRIMARY KEY,
    AssignmentID INT NOT NULL,
    ClaimDate DATE NOT NULL,
    ClaimAmount MONEY NOT NULL,
    ClaimStatus VARCHAR(20) NOT NULL,
CONSTRAINT FK_Claims_Assignments
        FOREIGN KEY (AssignmentID) REFERENCES PolicyAssignments(AssignmentID),

    );
```

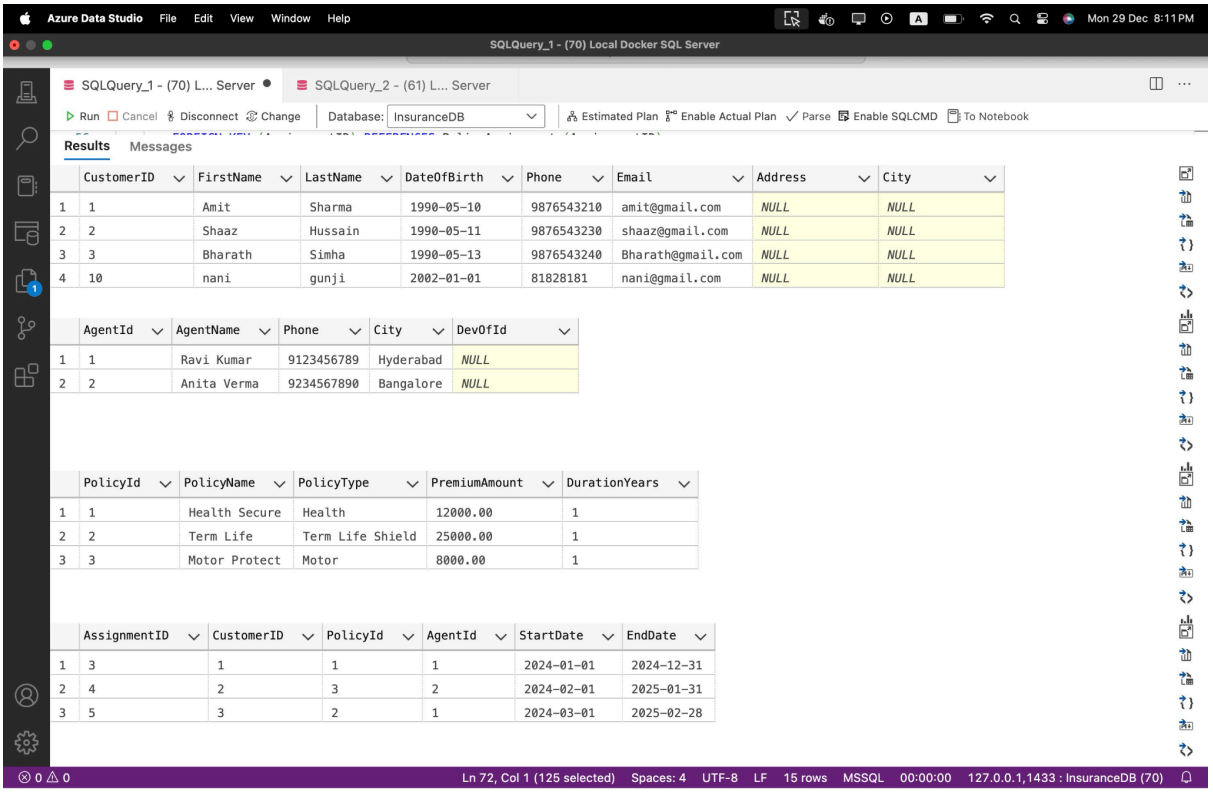
3. Insert commands for all tables.

```
INSERT INTO  Customers (FirstName,LastName,DateOfBirth,Phone,Email)
values('Amit','Sharma','1990-05-10','9876543210','amit@gmail.com');
INSERT INTO Customers  (FirstName,LastName,DateOfBirth,Phone,Email)
values('Shaaz','Hussain','1990-05-11','9876543230','shaaz@gmail.com');

INSERT INTO Customers (FirstName,LastName,DateOfBirth,Phone,Email)
values('Bharath','Simha','1990-05-13','9876543240','Bharath@gmail.com');

INSERT INTO Policies(PolicyName,PolicyType,PremiumAmount,DurationYears)
VALUES('Health Secure','Health',12000.00,1)
INSERT INTO Policies(PolicyName,PolicyType,PremiumAmount,DurationYears)
VALUES('Life Shield','Life',25000.00,1)
```

```sql
INSERT INTO Policies(PolicyName,PolicyType,PremiumAmount,DurationYears)
VALUES('Motor Protect','Motor',8000.00,1)
```

4. Select commands

```sql
select * from Customers;
select * from Agents;
select * from Policies;
select * from PolicyAssignments;
select * from Claims;
```

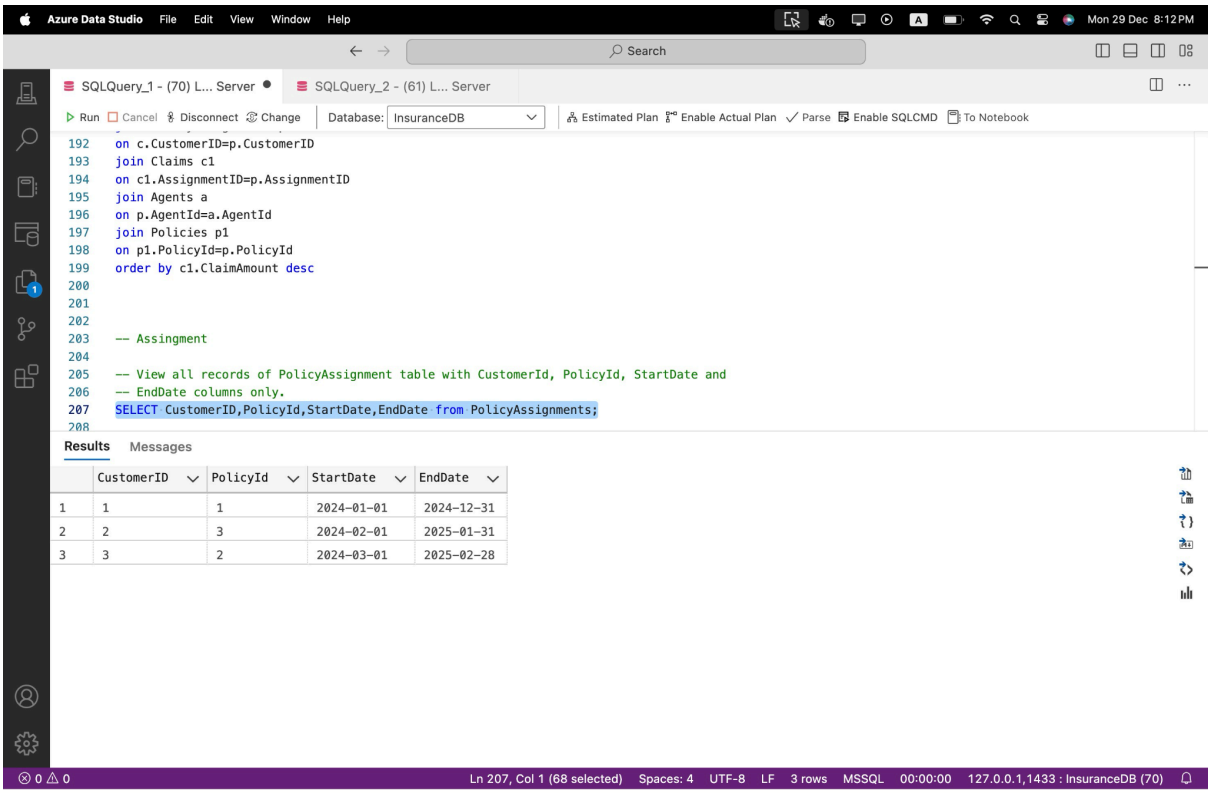


1. View all records Customers table.

```sql
select * from Customers;
```

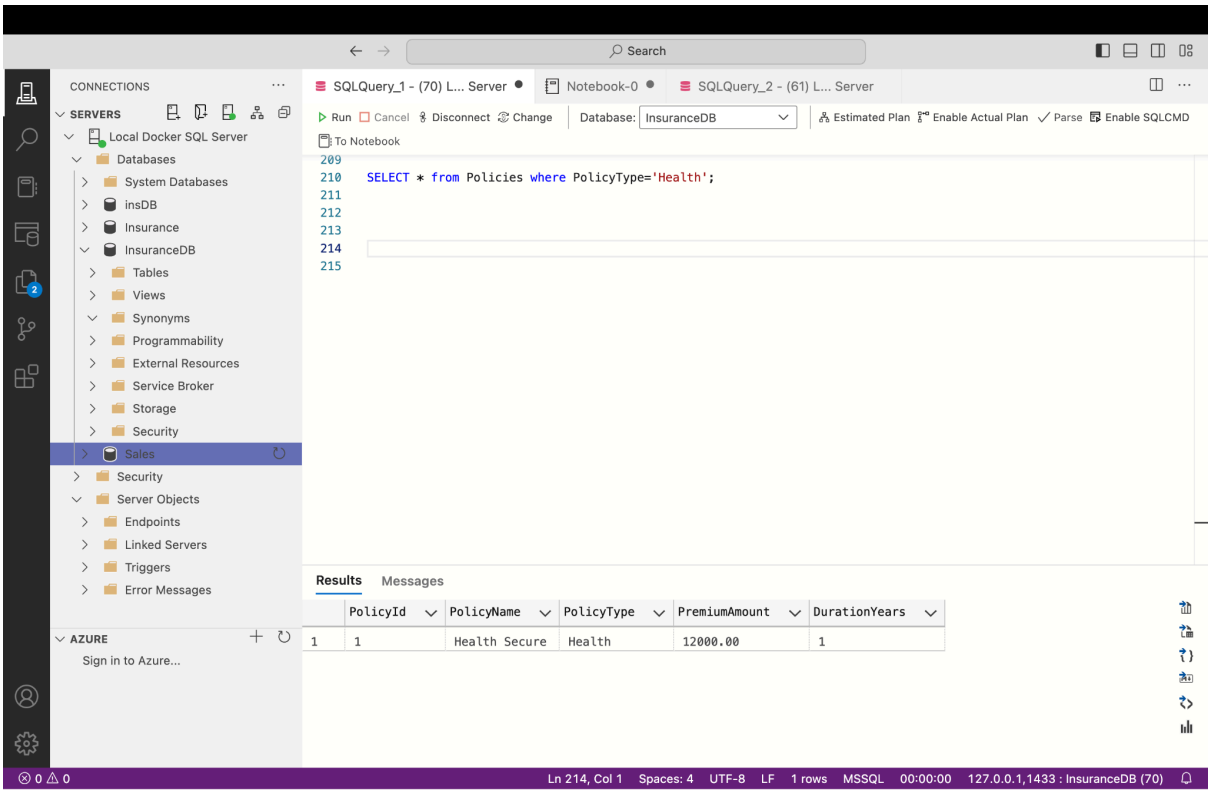
2. View all records of PolicyAssignment table with CustomerId, PolicyId, StartDate and EndDate columns only.

```
SELECT CustomerID,PolicyId,StartDate,EndDate from PolicyAssignments;
```



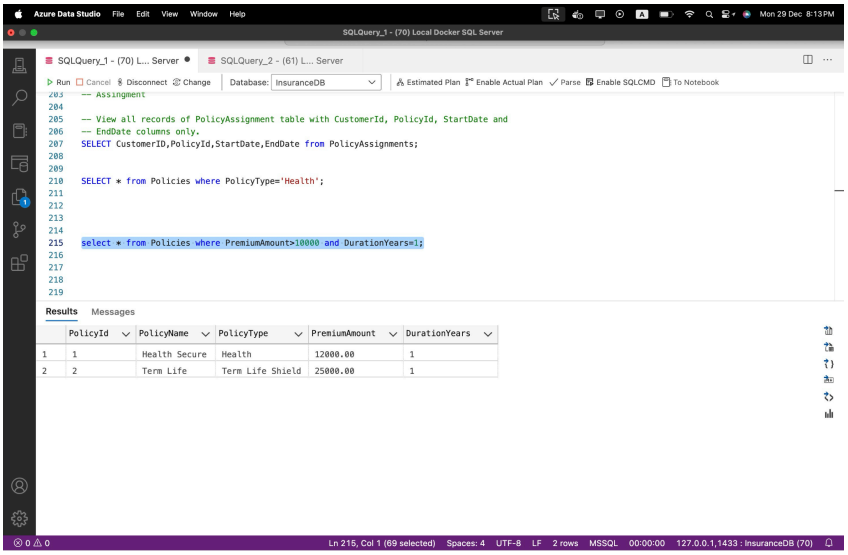
3. Display all policies of Health type.

```
SELECT CustomerID,PolicyId,StartDate,EndDate from PolicyAssignments;
```



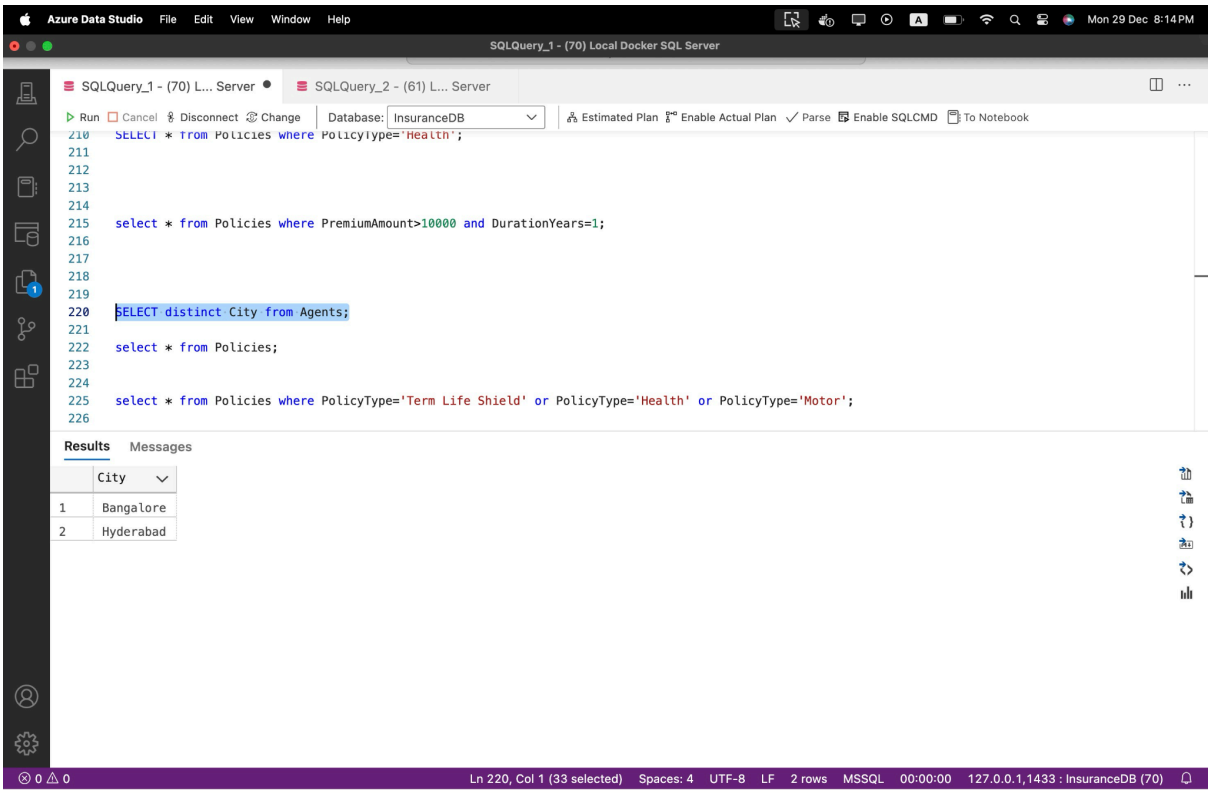
4. Display policies having premium amount more than 10000 and DurationYears is 1.

```
select * from Policies where PremiumAmount>10000 and
DurationYears=1;
```



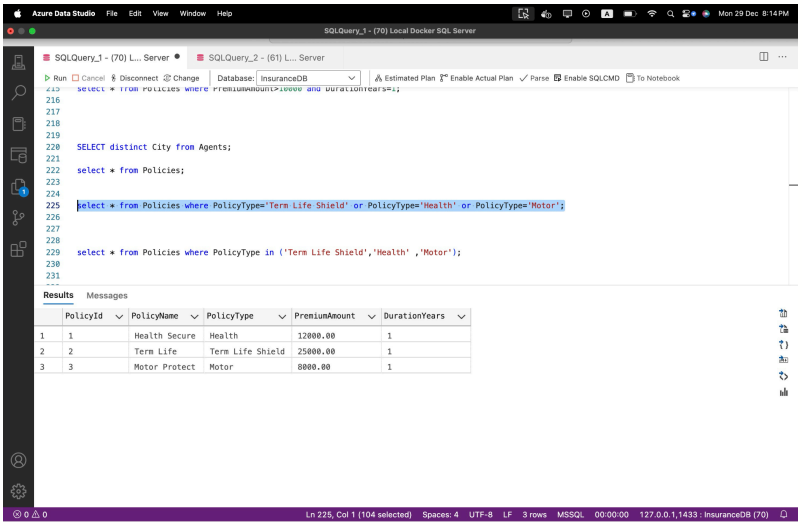
5. Display unique city names from where agents belong to.

```
SELECT distinct City from Agents;
```



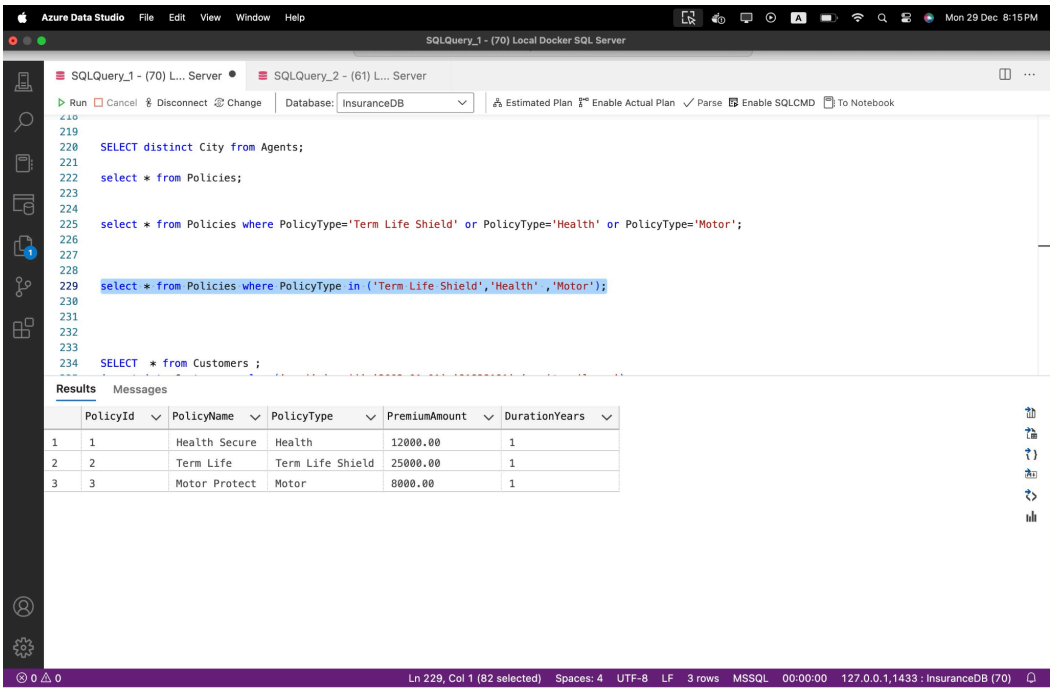
6. List policies of type Life, Health, Motor use OR clause.

```
select * from Policies where PolicyType='Term Life Shield'
or PolicyType='Health' or PolicyType='Motor';
```



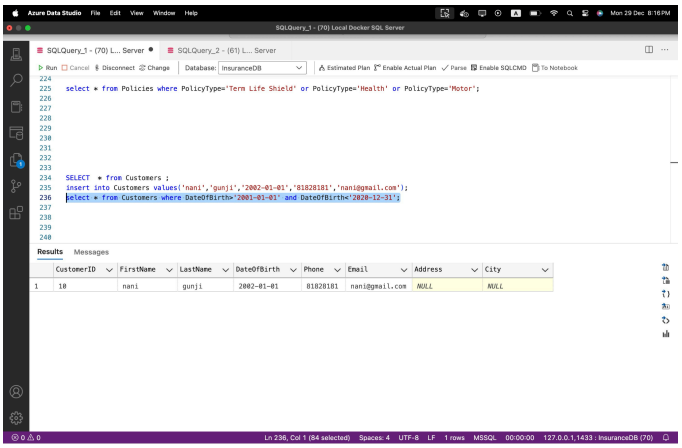
7. List policies of type Life, Health, Motor use IN operator.

```
select * from Policies where PolicyType in ('Term Life
Shield','Health' ,'Motor');
```



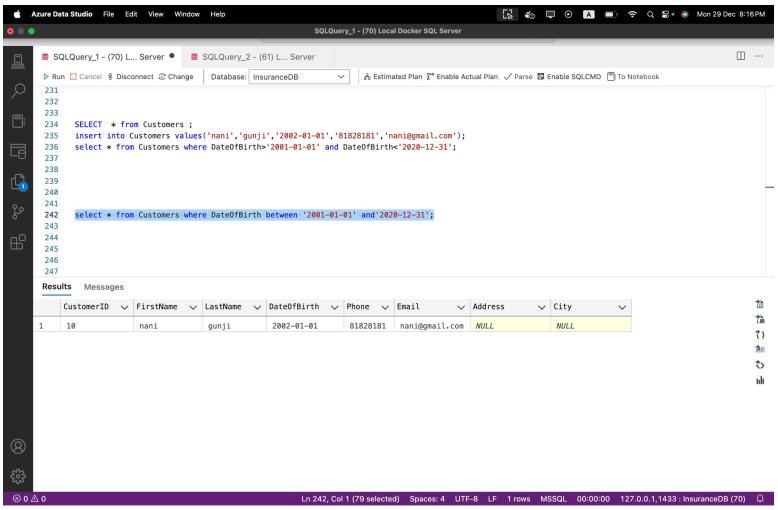
8. Display list of customers born after January 1 st , 2001 and before December 31 st , 2020 using >= and <= operators.

```
select * from Customers where DateOfBirth>'2001-01-01' and DateOfBirth<'2020-12-31';
```



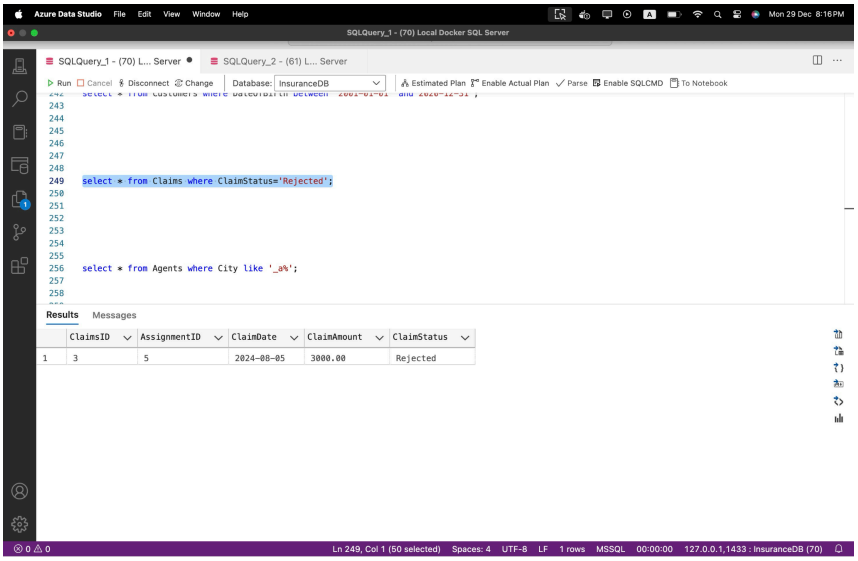
9. Display list of customers born after January 1 st , 2001 and before December 31 st , 2020 using between operator.

```
select * from Customers where DateOfBirth between '2001-01-01' and'2020-12-31';
```



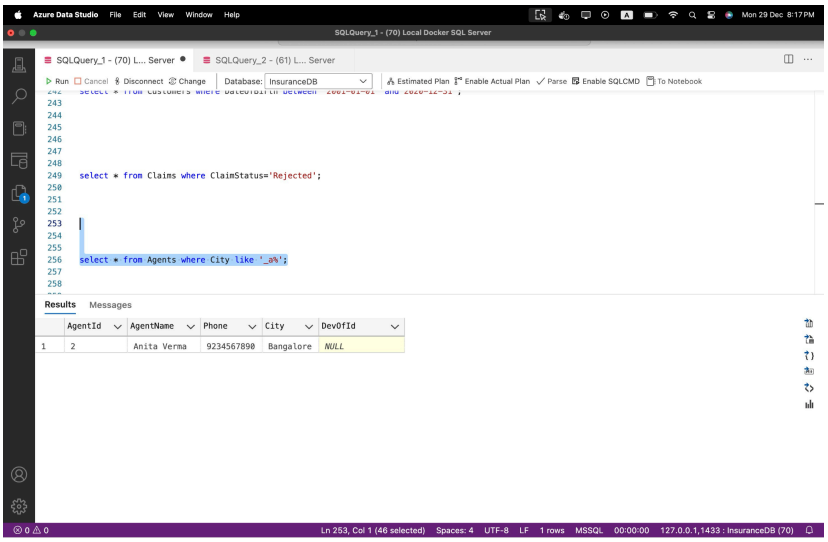
10. Display claims data where claim status is Rejected.

```
select * from Claims where ClaimStatus='Rejected';
```



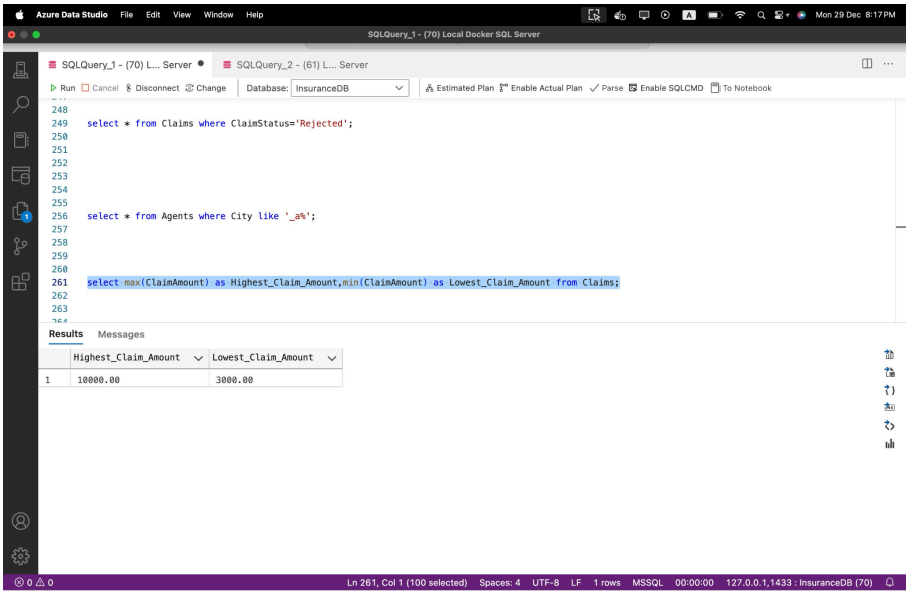
11. Display records of Agents who stay in a city whose second letter is 'a'.

```
select * from Agents where City like '_a%';
```



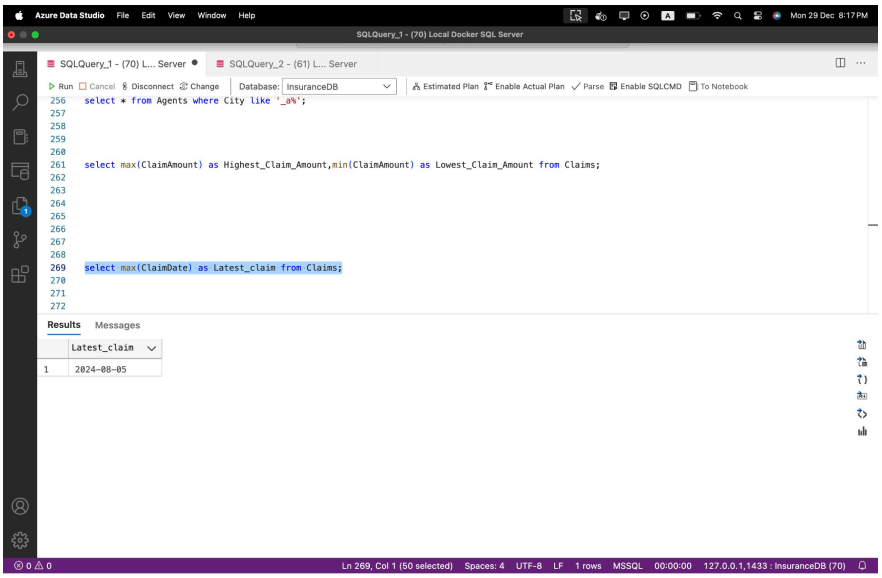
12. Display highest and lowest claimAmount from Claims table.

```
select max(ClaimAmount) as
Highest_Claim_Amount,min(ClaimAmount) as
Lowest_Claim_Amount from Claims;
```



13. Display latest claim record.

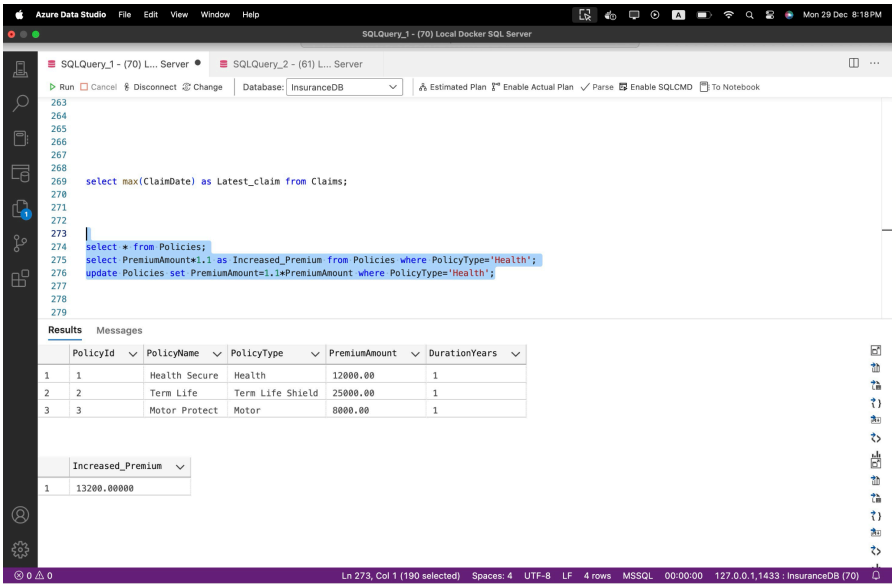
```
select max(ClaimDate) as Latest_claim from Claims;
```



14. Increase premium amount to 10% for all health insurance policies.

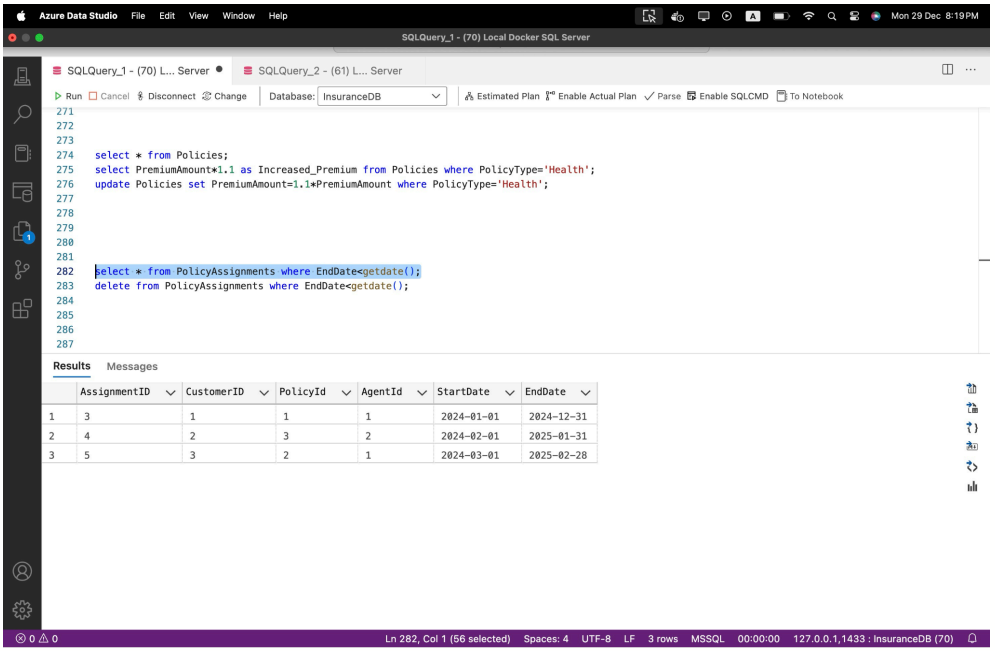
```
select PremiumAmount*1.1 as Increased_Premium from
Policies where PolicyType='Health';
```

```
update Policies set PremiumAmount=1.1*PremiumAmount where
PolicyType='Health';
```



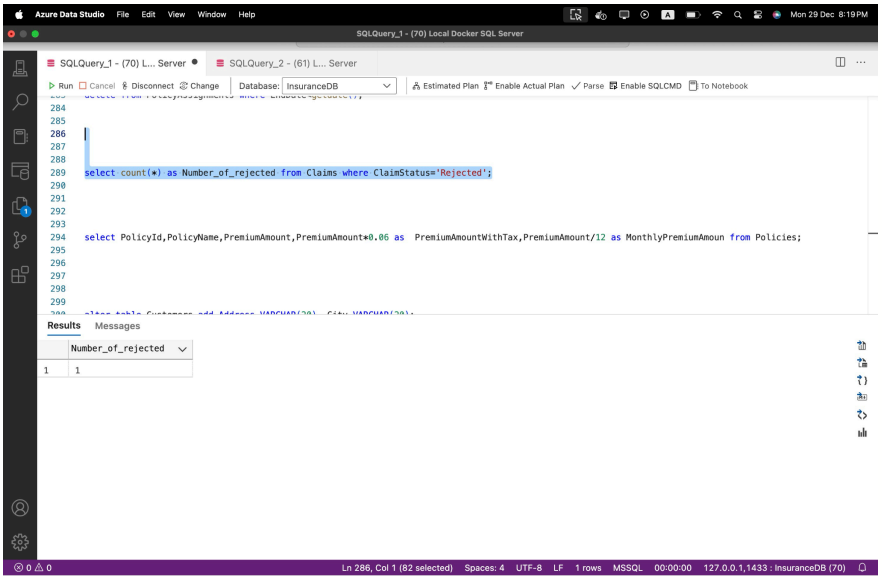
15. Delete the record of PolicyAssignments whose EndDate is before today's date.

```
select * from PolicyAssignments where EndDate<getdate();
delete from PolicyAssignments where EndDate<getdate();
```



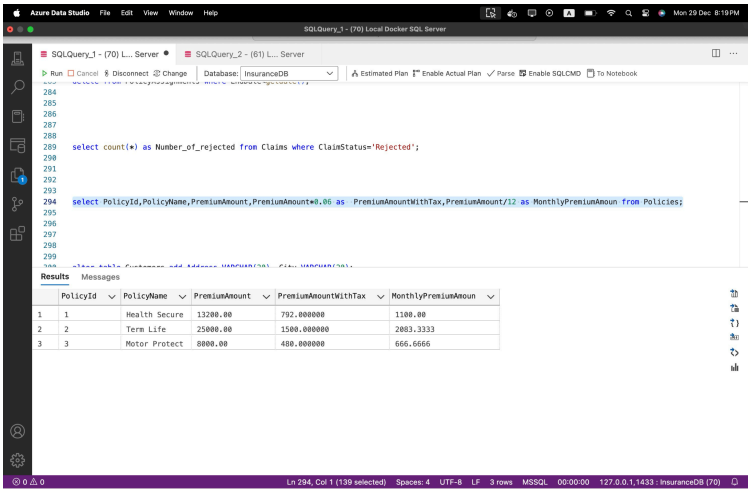
16. Display no of claims rejected.

```
select count(*) as Number_of_rejected from Claims where
ClaimStatus='Rejected';
```



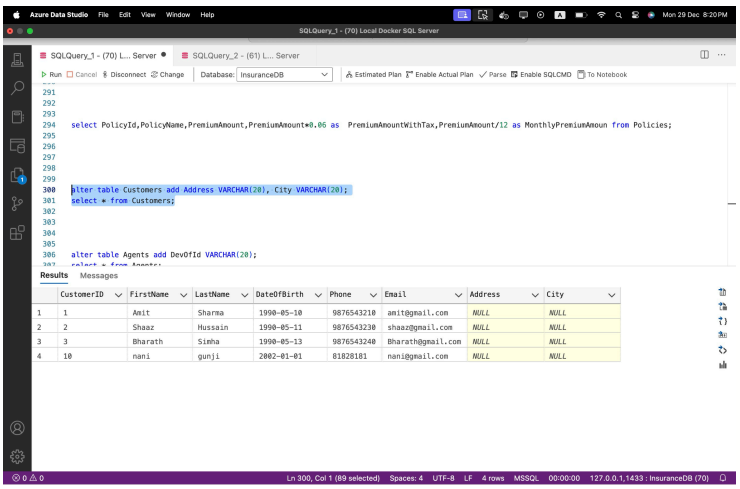
17. Display PolicyId, PolicyName, PremiumAmount along with computed fields not in table à 6% LocalTaxes, PremiumAmountWithTax and MonthlyPremiumAmount considering PremiumAmount is Annual.

```
select
PolicyId,PolicyName,PremiumAmount,PremiumAmount*0.06 as
PremiumAmountWithTax,PremiumAmount/12 as
MonthlyPremiumAmount from Policies;
```



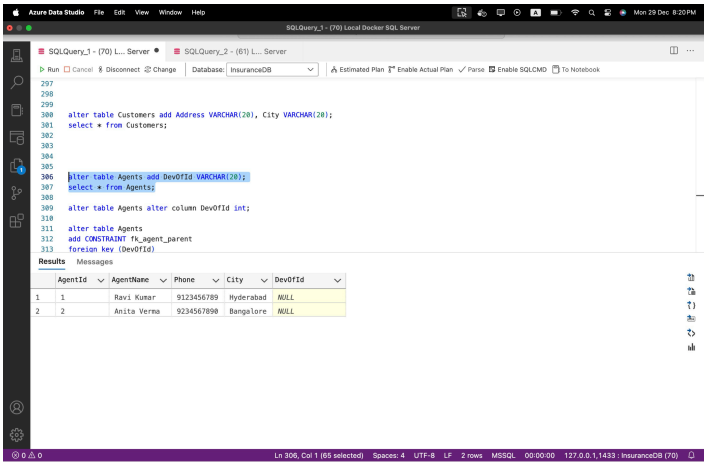
18. Write a command to add Address and City Columns in the Customers table.

```
alter table Customers add Address VARCHAR(20), City VARCHAR(20);
select * from Customers;
```



19. Write a command to add a new column named DevOfId (DevelopmentOfficerId) in an existing Agents table.

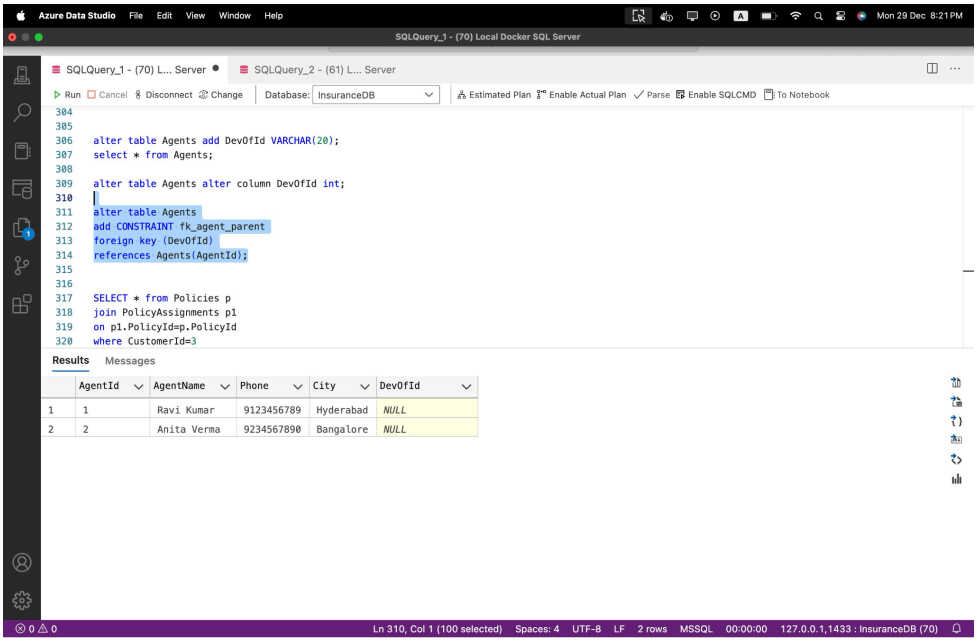
```
alter table Agents add DevOfId VARCHAR(20);
select * from Agents;
```



20. Write command to make the above DevOfId as a recursive foreign key to AgentId as Parent.

```
alter table Agents
add CONSTRAINT fk_agent_parent
```

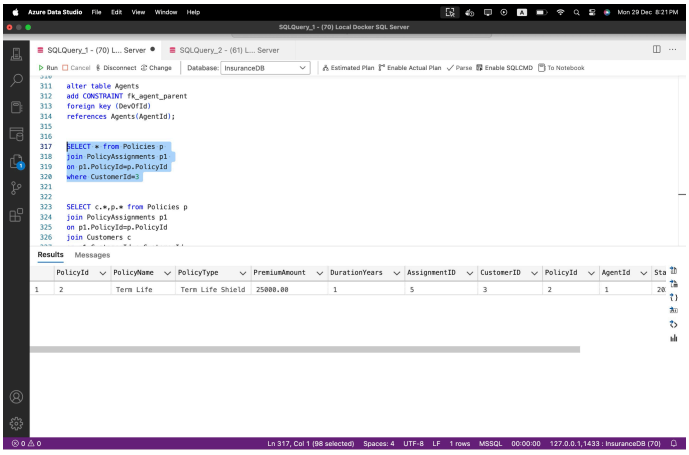
```
foreign key (DevOfId)
references Agents(AgentId);
```



5. Queries using Joins, Group By, Having etc.

1. List all Policies for a CustomerId 5.

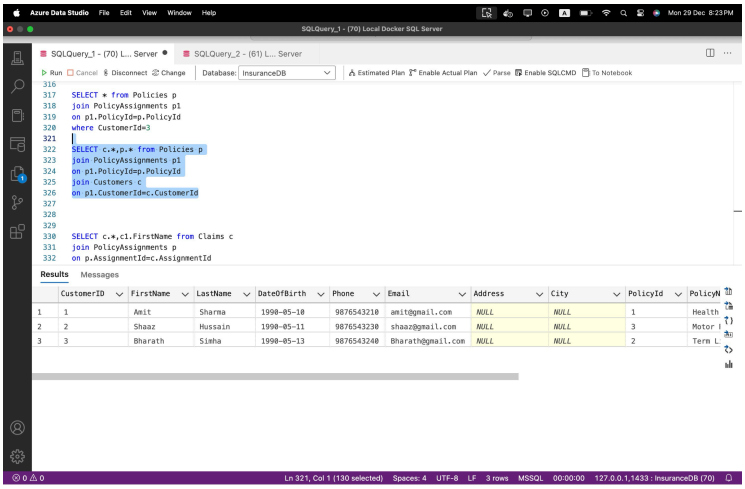
```
SELECT * from Policies p
join PolicyAssignments p1
on p1.PolicyId=p.PolicyId
where CustomerId=3
```



2. View all customers with their policies.

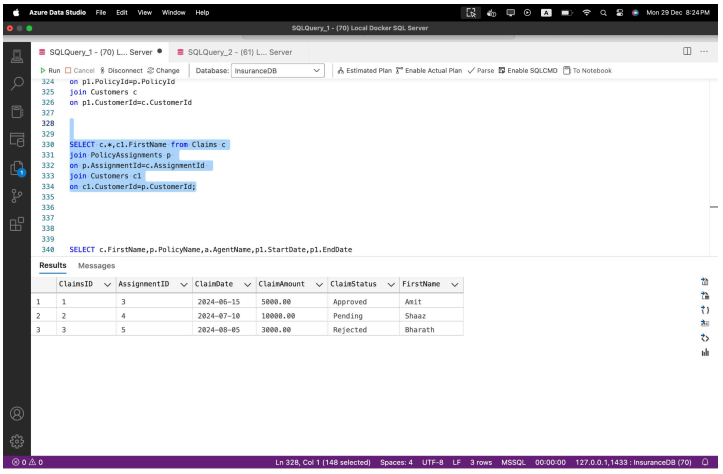
```
SELECT c.*,p.* from Policies p
join PolicyAssignments p1
```

```
on p1.PolicyId=p.PolicyId
join Customers c
on p1.CustomerId=c.CustomerId
```



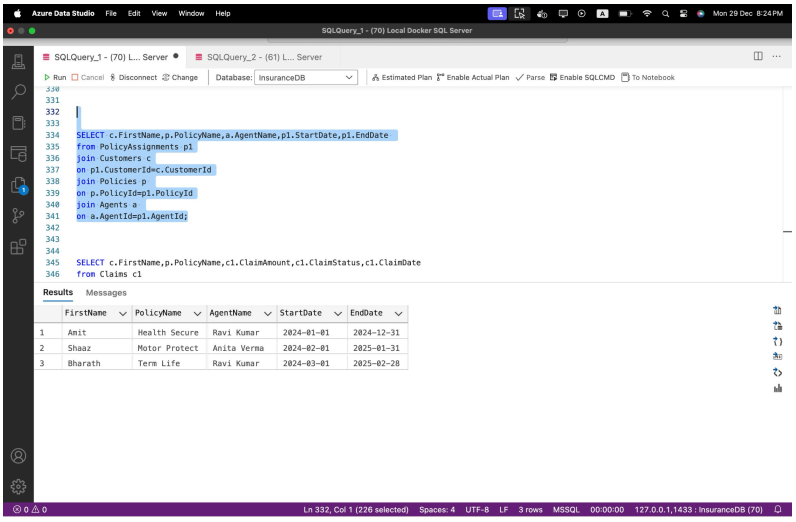
3. View claims with customer name.

```
SELECT c.*,c1.FirstName from Claims c
join PolicyAssignments p
on p.AssignmentId=c.AssignmentId
join Customers c1
on c1.CustomerId=p.CustomerId;
```



4. Display FirstName, PolicyName, AgentName, StartDate and EndDate from their respective tables.

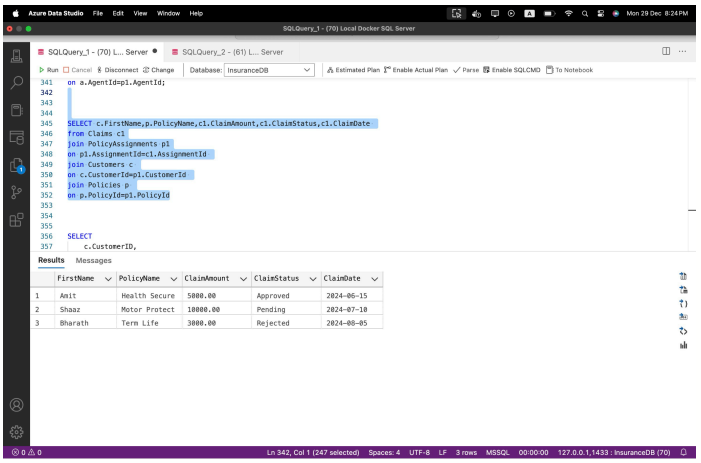
```
SELECT
c.FirstName,p.PolicyName,a.AgentName,p1.StartDate,p1.EndDa
te
from PolicyAssignments p1
join Customers c
on p1.CustomerId=c.CustomerId
join Policies p
on p.PolicyId=p1.PolicyId
join Agents a
on a.AgentId=p1.AgentId;
```



5. Display claims report with FirstName, PolicyName, ClaimAmount, ClaimStatus, and ClaimDate from their respective tables.

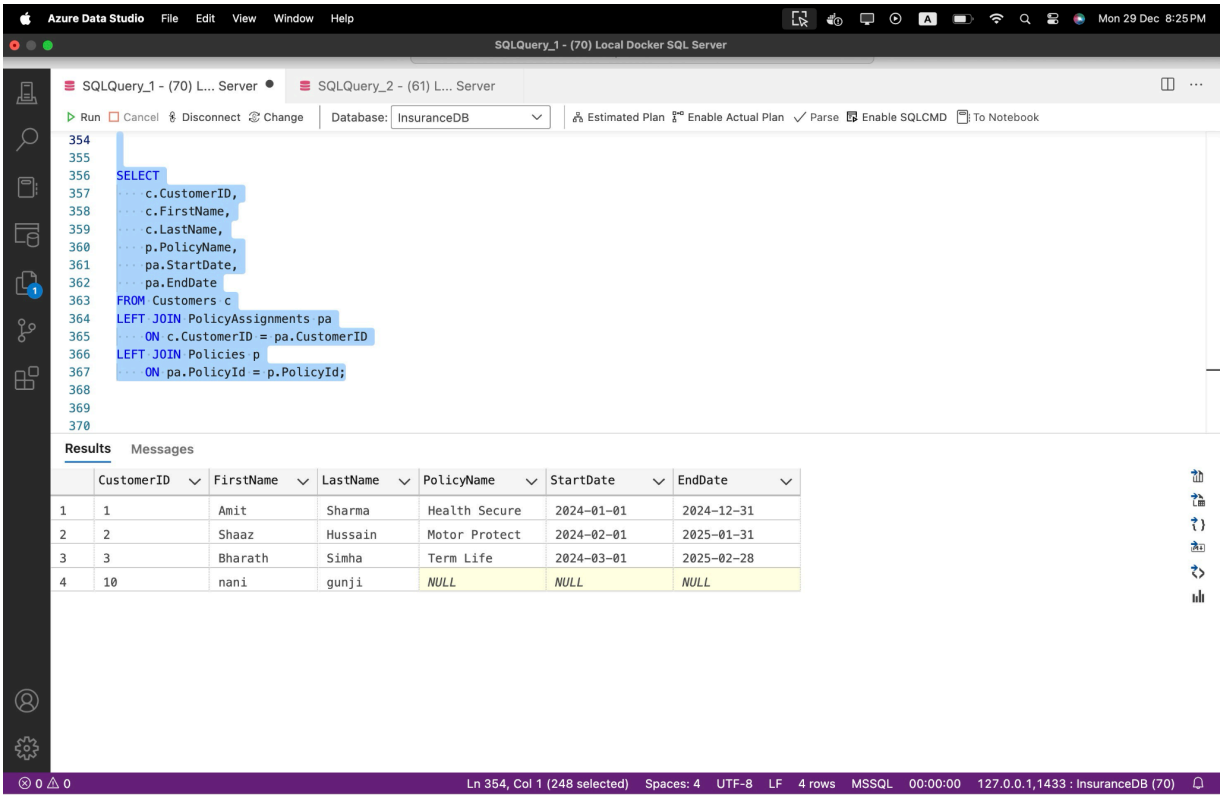
```
SELECT
c.FirstName,p.PolicyName,c1.ClaimAmount,c1.ClaimStatus,c1.
ClaimDate
from Claims c1
join PolicyAssignments p1
on p1.AssignmentId=c1.AssignmentId
join Customers c
on c.CustomerId=p1.CustomerId
join Policies p
```

```
on p.PolicyId=p1.PolicyId
```



6. Display records of Customers with or without Policies.

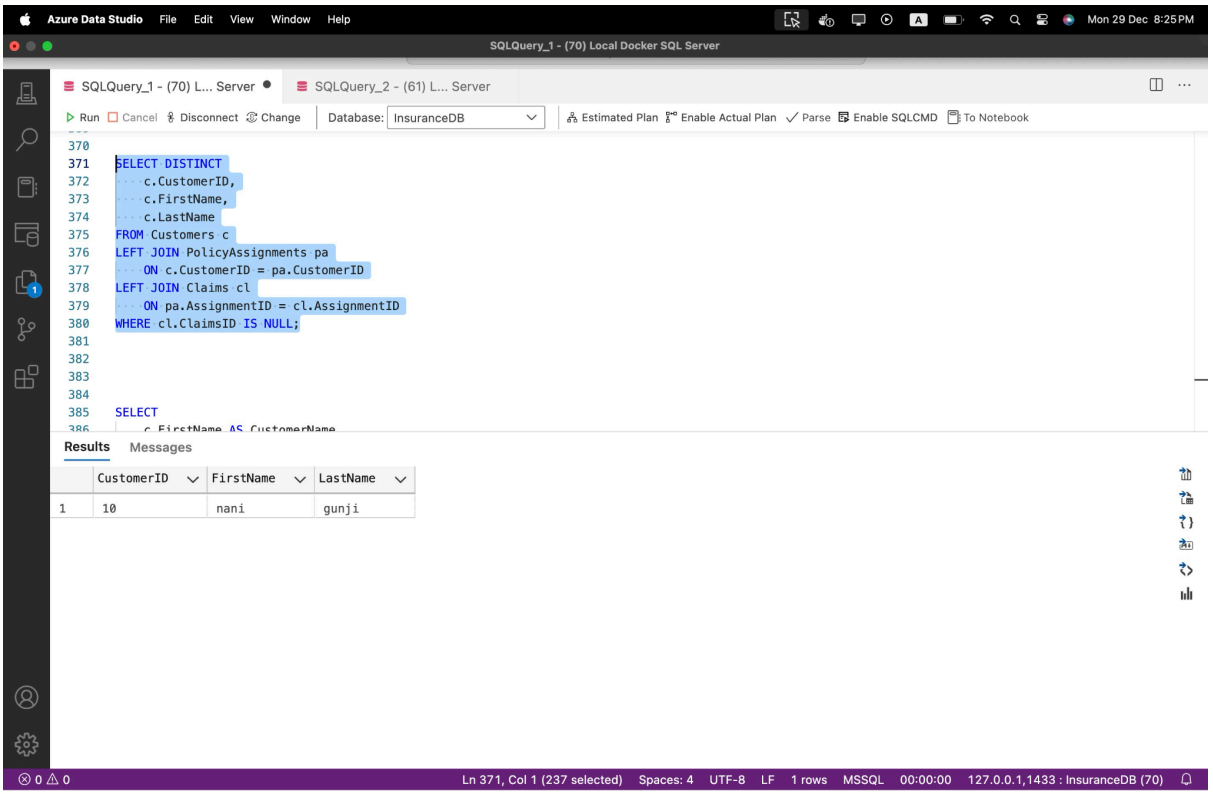
```
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    p.PolicyName,
    pa.StartDate,
    pa.EndDate
FROM Customers c
LEFT JOIN PolicyAssignments pa
ON c.CustomerID = pa.CustomerID
LEFT JOIN Policies p
ON pa.PolicyId = p.PolicyId;
```



7. Display all Customers with NO Claims.

```sql
SELECT DISTINCT
    c.CustomerID,
    c.FirstName,
    c.LastName
FROM Customers c
LEFT JOIN PolicyAssignments pa
    ON c.CustomerID = pa.CustomerID
LEFT JOIN Claims cl
    ON pa.AssignmentID = cl.AssignmentID
```

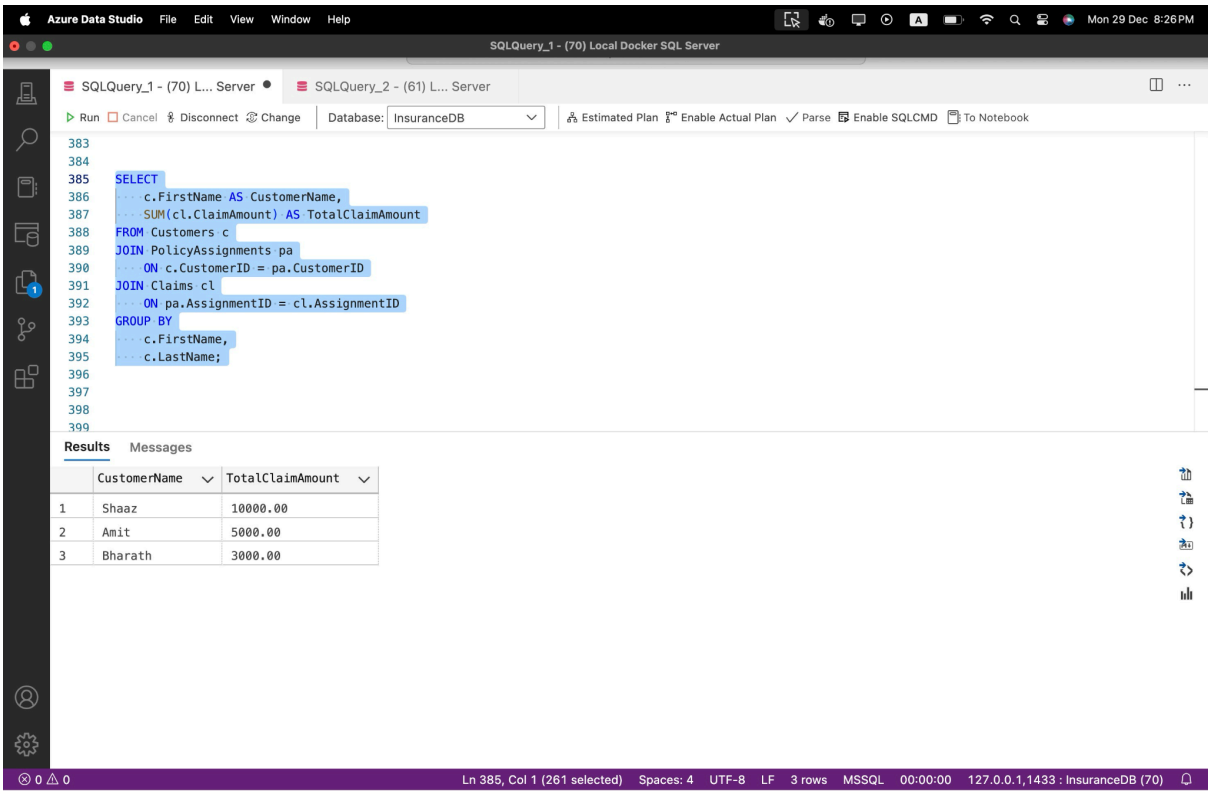
```
WHERE cl.ClaimsID IS NULL;
```



8. Show CustomerName with Total Claim Amount per Customer.

```
SELECT
    c.FirstName AS CustomerName,
    SUM(cl.ClaimAmount) AS TotalClaimAmount
FROM Customers c
JOIN PolicyAssignments pa
    ON c.CustomerID = pa.CustomerID
JOIN Claims cl
    ON pa.AssignmentID = cl.AssignmentID
GROUP BY
    c.FirstName,
```

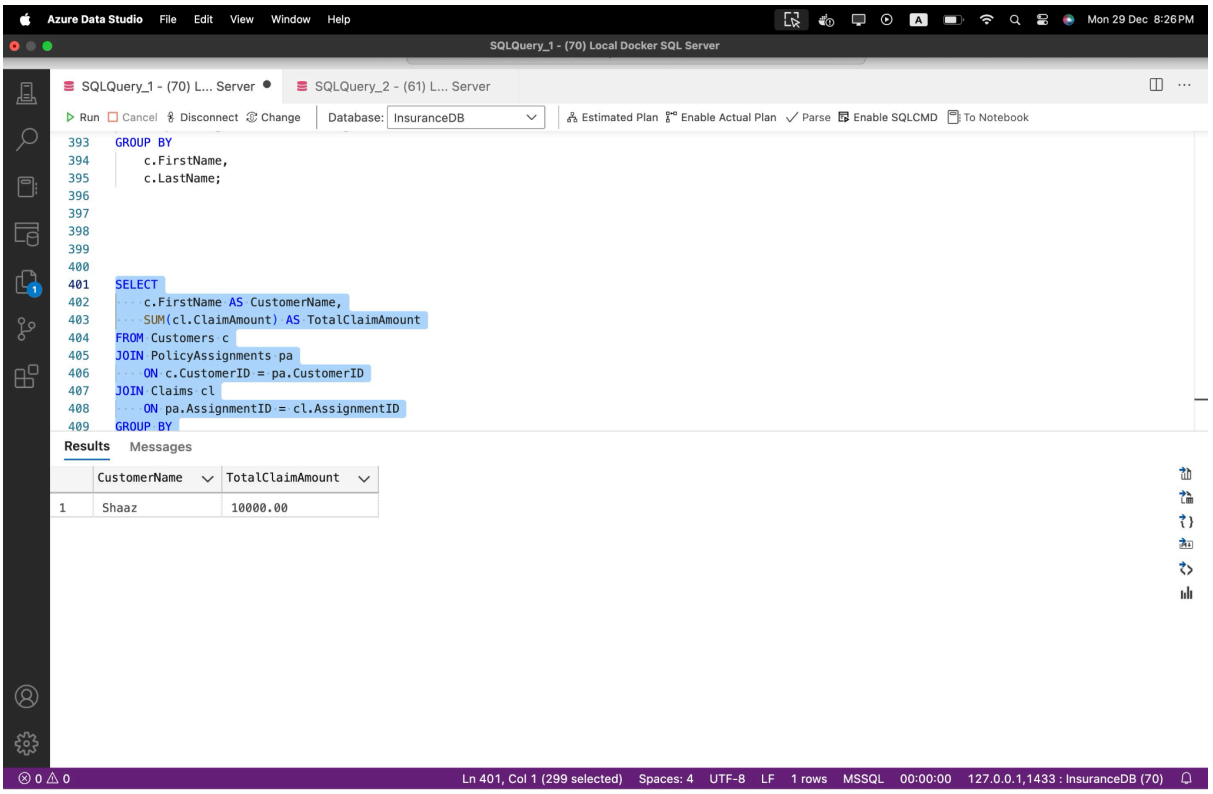
```
    c.LastName;
```



9. Show names and total claim amount of Customers With Claim Amount > 50000 (Use HAVING Clause).

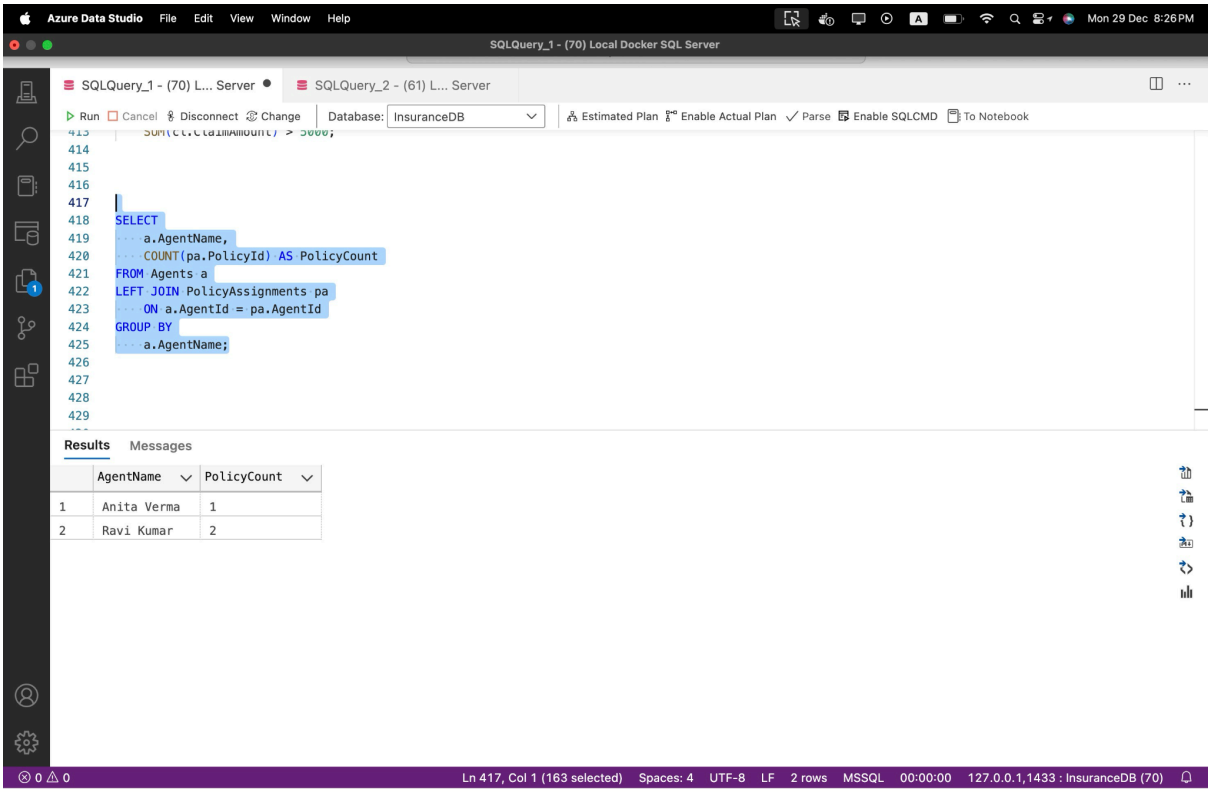
```
SELECT
    c.FirstName AS CustomerName,
    SUM(cl.ClaimAmount) AS TotalClaimAmount
FROM Customers c
JOIN PolicyAssignments pa
    ON c.CustomerID = pa.CustomerID
JOIN Claims cl
    ON pa.AssignmentID = cl.AssignmentID
GROUP BY
```

```
    c.FirstName,
    c.LastName
HAVING
    SUM(cl.ClaimAmount) > 5000;
```



10. Display list with Agent Wise Policy Count.

```
SELECT
    a.AgentName,
    COUNT(pa.PolicyId) AS PolicyCount
FROM Agents a
LEFT JOIN PolicyAssignments pa
    ON a.AgentId = pa.AgentId
GROUP BY
    a.AgentName;
```


Azure Data Studio File Edit View Window Help
Mon 29 Dec 8:26 PM
SQLQuery_1 - (70) Local Docker SQL Server
SQLQuery_1 - (70) L... Server
SQLQuery_2 - (61) L... Server
Run Cancel Disconnect Change
Database: InsuranceDB
Estimated Plan Enable Actual Plan Parse Enable SQLCMD To Notebook
SELECT
a.AgentName,
COUNT(pa.PolicyId) AS PolicyCount
FROM Agents a
LEFT JOIN PolicyAssignments pa
ON a.AgentId = pa.AgentId
GROUP BY
a.AgentName;
Results
Messages
AgentName
PolicyCount
1 Anita Verma 1
2 Ravi Kumar 2
Ln 417, Col 1 (163 selected) Spaces: 4 UTF-8 LF 2 rows MSSQL 00:00:00 127.0.0.1,1433 : InsuranceDB (70)